



A tool to generate a **Snap4City** distribution
on a Docker platform

User manual + developer notes

Version 1.0, 2026-06-16

Index

Installing and Starting Snap4City Generator	3
Using the generator	5
Filling the placeholders	7
Using the configuration	9
Endpoints of python backend	13
/	13
'GET', POST	13
/updating_information	13
GET	13
/micro_components_x	13
GET	13
/normal_components_x_y	13
GET	13
/update_the_db_after_starting_it	13
GET	13
/updating_db	13
'GET'	13
/done	13
GET	14
/small_components_x_y	14
GET	14
/dcs_components_x_y_z	14
GET	14
/make_more_opensearch	14
POST, GET	14
/make_more_nifi	14
POST, GET	14
/more_nifi	14
GET	14
/more_opensearch	14
GET	14
/dcm_components_x_y_z_w	14
GET	15
/selecting_model	15
GET	15
/crash	15
GET	15
/kubernetes_configuration	15

GET	15
/fill_placeholders	15
POST, GET	15
/deal_with_placeholders	15
POST	15
/get_updating_tools	15
POST, GET	15
/saving_conf	15
POST	16
/conf_exists	16
POST	16

Installing and Starting Snap4City Generator

If for some goddamn reason you are installing your own generator instead the one accessible on snap4city.org, be sure to

- 1) Have docker installed on your host: <https://www.docker.com/>
- 2) Setup the docker-compose file to resolve the environment variables

```
services:
  app:
    container_name: app
    image: fabriziomereu/snap4city_installation_generator:rel.2.1.2
    links:
      - db
    ports:
      - "80:80"
    environment:
      - PYTHONUNBUFFERED
      - help_mail
      - send_placeholders
      - kuber
      - version
      - allow_compress
      - add_utils
      - send-aws-k8s
      - allow_update
      - MYSQL_ROOT_PASSWORD
      - time_wait_update_db
    command: >
      bash -c 'waitress-serve --port 80 --call flask_app:create_app'
    volumes:
      - ./Mysql/latest.sql:/data/mysql/a.sql
      - ./Output:/Flask/Output:rw
  db:
    container_name: db
    image: mysql:latest
    ports:
      - "32000:3306"
    command: mysqld --sql_mode=""
    cap_add:
      - SYS_NICE # CAP_SYS_NICE
    environment:
      - MYSQL_USER
      - MYSQL_PASSWORD
      - MYSQL_ROOT_PASSWORD
      - MYSQL_ROOT_HOST
    volumes:
      - container-volume:/var/lib/mysql
      - ./Mysql/latest.sql:/docker-entrypoint-initdb.d/a.sql
      - ./Mysql/users.sql:/docker-entrypoint-initdb.d/b.sql
volumes:
  container-volume:
```

- 3) Edit the ports in case they need to be different due to your host requirements

Then start the containers. The generator will become available in a few seconds after turning on the container as it sets up itself. The main page will be available at http://localhost/selecting_model

Make configuration - Docker

Problems? Send mail to support

Guide and examples

Ports and connections in models

Additional resources

version

Select model

-

Already have a token? Just put it here and resume from where you saved it. If your token is invalid you will not leave this webpage.

Use token

Select model

-

-

Micro X

Normal X, Y

Small X, Y

DataCitySmall X, Y, Z

DataCityMid X, K, S, W (draft)

DataCityLarge X, R, M, K, S, W (draft)

Kubernetes from Micro (experimental/draft)

Select model

Micro X

of IoT-Apps

3

IP addresses must be in the ipv4 format, and each one must be different from the other.
You will not be able to proceed with the configuration otherwise.

IP address of 1st machine

127.0.0.1

Fill placeholders

Note: If you have a token of a previously generated configuration, instead use the form on the right.

Already have a token? Just put it here and resume from where you saved it. If your token is invalid you will not leave this webpage.

Use token	someexadecimalcode
-----------	---

Filling the placeholders

Base access link for the whole service.

`$$base-hostname$$` - string

demonstration

Choose between HTTP and HTTPS (HTTPS requires the certificates)

`$$base-protocol$$|$$dashboard-websocket-protocol$$` - select

http|ws

Remember to execute the .sh files when deploying the applications; setup.sh must be ran before execution, post-setup.sh must be ran during execution. Multiple executions are safe. Some folders may contain no such files; in that case, there is nothing you need to do.

In order to access Nifi securely, we had to use a Certificate, which is actually generated during the setup. You'll need to allow the certificate, or else you will receive an insecure connection warning.

You can't download a configuration until you save it.

Save configuration

Show default fields

Download configuration

First, choose the hostname. The host should be able to resolve this name as itself. Then choose between http and https. Choosing http results in a simpler configuration, and choosing https results in a more robust solution in terms of security. By clicking on the "Show default fields" you will be shown a number of preconfigured placeholders. Some of them are randomly generated (*not cryptographically secure*), while others have default values. Edit them at your own risk.

Model name

Micro

of IoT-Apps

3

of IoT-Brokers

1



Latitude broker-0

41.40318532580696

Longitude broker-0

38.116070649804314

ip_field-1

127.0.0.1

Save this token to load the current configuration in the future.

Time of configuration's last saving

On the second part of the configurator (on the right) you will see the configuration which depends on the model of the distribution. In this example we are shown that we picked 3 IoTAPPs, we have 1 IoT-Broker for which we have to pick the coordinates, we are reminded that the IP of the first (and the only one, in this scenario) host is 127.0.0.1. The latter 2 fields represent information which depends on the operation saving of the configuration.

Before downloading your configuration, you must save it first. After saving it, the server will allow you to download your configuration; additionally you will receive a token to resume the configuration in the future. Once you download the generation, a few seconds will pass before you receive a file; the server is elaborating all the changes it needs to perform to provide you with your configuration. You will eventually receive a zip file containing the configuration. The title of the file represents the configuration and includes the token so that, as long as you don't rename the archive, you can always retrieve the token.

Using the configuration

From the previous step, you should have obtained a zip file with your configuration. The is named like the following:

version-model-timeisostring-parameters-token-config

With version indicating which version of the generator created this configuration, model indicating the model of the Snap4City distribution (in this example it will be MicroX), the isostring represents the timestamp of the creation of the configuration, the parameters represents the variables of the configuration (in MicroX, it will be the amount of the IoTAPPs), the token indicates the hexadecimal string which allows you to retrieve your distribution from the generator.

As an archive you'll have to decompress it. The files should be placed on the host which has the indicated IP address (a folder named checker represents Snap4Sentinel, which is covered in another document). On a MicroX there is a single host, but in other models you'll have more. You will also find a file which shows you all the placeholders used to generate your configuration.

Once all the files are in the correct place, the correct sequence of actions to get the distribution running is the following:

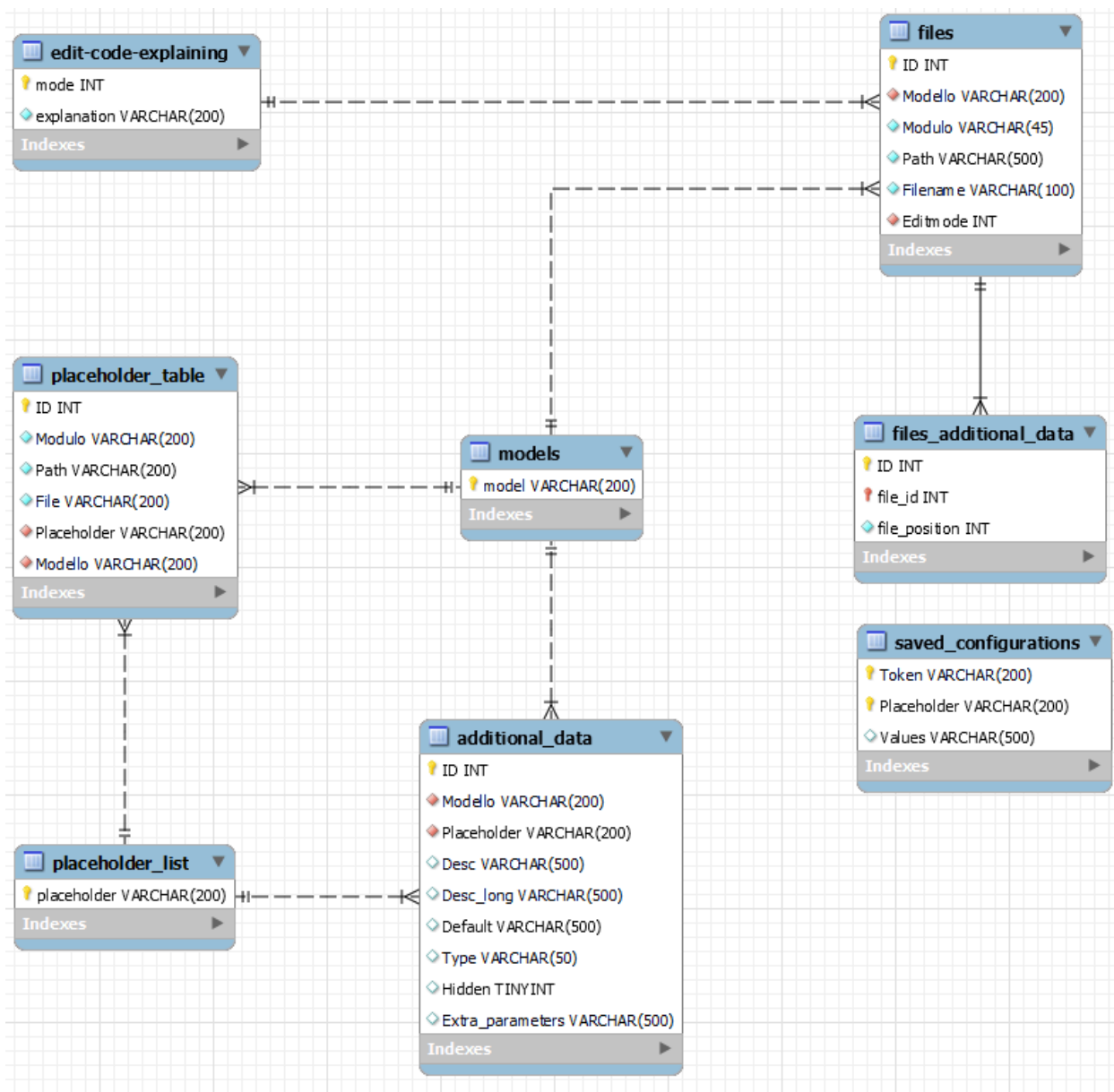
- 1) Ensure to have docker and docker compose installed.
 - 2) Ensure to have python 3.9 installed.
 - 3) Run the **setup.sh** script, found inside the folder named as the IP.
 - 4) In the console, for each host, run **docker compose up -d**, which starts the containers then detaches their output from the console, allowing you to perform other commands
 - 5) Wait 60-90 seconds for the containers to stabilize. Depending on your hardware, you may need to wait more time.
 - 6) Run the **post-setup.sh** script, in the same location as the other script(s).
- After following the previous steps, the distribution will be up and running.

Internal infrastructure

The generator has two components; a python frontend+backend and a mysql database. On load of the database, a user with limited privileges is created and the python server will use that user to access/write its data to the db. The python backend will try to update the database structure on startup (waiting a few seconds), so that if a new update of the generator is released, it will populate the database automatically.

During the development of this tool, some initial assumptions were later found to be not needed. It is therefore possible that some functionalities do not make sense anymore.

Database infrastructure



saved_configurations: stores saved configurations so that someone may retrieve it later.

files_additional_data: used only when a docker configuration is distributed across multiple hosts (so, not a microx and not a kubernetes). The file position field means which n-th host the file with the given id should go to. A file that is not listed here is assumed to have a position of 0, which means "the first host".

files: model means MicroX, Normal, etc. Modulo is a leftover column not actually used for anything. Path and filename indicate where the file is and what that file is, and what it will be and where it will be in the configuration when one is generated. Editmode flags how the file should be handled when replacing placeholders; 1 means do not edit, 2 means special generation and 3 means replace placeholders.

models: This table lists which models are valid. Note that these are detached from the ones you see in the main ui, so you have to edit both if you need to apply a patch of some sort.

edit-code-explanation: explains with words what the integers mean in the other table.

placeholder_list: it's the list of possible placeholders used. Their syntax is `$$string#$$` or `$$string1#$$string2#$$`. Note that elsewhere they may appear as a combination of these. Adding a new placeholder means adding it here first, or else the rest of the database won't accept it.

placeholder_table: this table isn't really used anymore.

additional_data: this table indicates which values the placeholders should have. The browser UI pulls the data from this table.

ID	Modello	Placeholder	Desc	Desc_long	Default	Type	Hidden	Extra_parameters
98	Normal	\$\$dashboard-cross-...	How much time ...	The default value is fine a...	1000	number	0	NULL
796	Kubernetes	\$\$orion-default-port...	The default port...	Don't assign the same por...	1026	number	1	NULL
450	DataCitySmall	\$\$personaldata-db-...	NULL	NULL	\$\$dashboard-db-pwd#\$\$	password	0	NULL
663	DataCityLarge	\$\$ldap-admin-pwd#\$\$	Password for th...	'secret' is the default as it...	secret	password	0	16
1047	Normal	\$\$areamanager-pw...	Password for us...	This password refers to u...	replaceme	password	1	16
897	DataCityMedium	\$\$ldap-port#\$\$	NULL	NULL	\$\$ldap-server-port#\$\$	port	0	NULL
1003	Normal	\$\$ws-port#\$\$	Port for the web...	Best left at 80	80	port	1	NULL
838	Kubernetes	\$\$aes-encryption-iv...	Encryption value...	16 random chars is usuall...	IVKeyivKey123456	random	0	16
1010	Kubernetes	\$\$send-aws-k8s#\$\$	Do you want th...	Requires additional setup ...	No	select	0	No,Yes
909	DataCityMedium	\$\$orion-protocol#\$\$	The current mod...	Https is currently under d...	http	select	1	http,https
908	DataCityMedium	\$\$base-protocol#\$\$	Https is work in ...	Eventually, it will be free ...	NULL	select	1	http ws,https wss
903	DataCityMedium	\$\$elasticsearch-sch...	ElasticSearch's s...	It is not recommended to ...	https	select	1	https,http
77	Normal	\$\$ldap-organization#\$\$	Main value for t...	This is a standard value t...	organization	string	0	NULL
1033	Micro	\$\$sentinel-user-psw...	Password for th...	The relative has is create...	VG5pDC3tkxqkSi14jSG	string	0	16
1073	Kubernetes	\$\$main-menu-name#\$\$	Name on top of ...	This string will appear on ...	Snap4CityDocker	string	0	20
939	Small	\$\$orion-host#\$\$	The host of Orion.	The default is orion<...	\$\$ip-3#\$\$	string	1	NULL
692	DataCityLarge	\$\$personaldata-db-...	NULL	NULL	\$\$dashboard-db-host#\$\$	url	0	NULL

Above there are sample rows from the db. We can see examples of referenced placeholders (ID 692, column Default), unset values (ID 908, column Default), many types, the hidden property (will or will not shown as default in main UI), how to populate some parameters (logic is in frontend, for instance digits are length, strings separated by “,” are options). Note that a referenced placeholder will not show up at all in the main UI, as these values wouldn't work if they were different from the thing they are referencing.

Endpoints of python backend

/

GET

Goes to an unused endpoint.

/updating_information

GET

Shows an html explaining how to update an existing microx

/micro_components_x

GET

Shows an html explaining how the components of a microx are connected to each other.

/normal_components_x_y

GET

Shows an html explaining how the components of a normalxy are connected to each other.

/update_the_db_after_starting_it

GET

Opens an UI to manually update the database's data.

/updating_db

GET

Endpoint to manually update the db.

/done

GET

Endpoint to tell the user a certain operation was performed

/small_components_x_y

GET

Shows an html explaining how the components of a smallxy are connected to each other.

/dcs_components_x_y_z

GET

Shows an html explaining how the components of a datacityxyz are connected to each other.

/make_more_opensearch

POST, GET

Generates a configuration for a multinode opensearch cluster, not really used. As a post, returns that configuration.

/make_more_nifi

POST, GET

Generates a multi node nifi configuration (probably old at this point), not used in production.

/more_nifi

GET

Opens a webpage which should show how to generate a multi node nifi cluster configuration, nor used in production.

/more_opensearch

GET

Opens a webpage which should show how to generate a multi node opensearch cluster configuration, nor used in production.

/dcm_components_x_y_z_w

GET

Shows an html explaining how the components of a datacitymediumxy are connected to each other.

/selecting_model

GET

Main page of the UI.

/crash

GET

Intentionally fails, as a test (unused)

/kubernetes_configuration

GET

Redirects to main form

/fill_placeholders

POST, GET

Main form

/deal_with_placeholders

POST

Actual logic for generating a configuration

/get_updating_tools

POST, GET

Sends an archive with the files commonly used to update a configuration (hasn't been used recently)

/saving_conf

POST

Saving a configuration's endpoint.

/conf_exists

POST

Endpoint to determine if an configuration exists